

Low-Latency Close-Range Drone Detection with Quantized YOLOv8 on a Zynq UltraScale+ MPSoC

Alex Naumov

Independent researcher

beatwad@gmail.com

Abstract

We describe a complete vision chain for a close-range (≤ 10 m), low-latency drone detection system for aim assistance. It runs on a Xilinx ZCU102 board and outputs the target’s angular offset to a downstream pointing subsystem. The detector is a single-head, stride-8 variant of YOLOv8n with ReLU6 activations, trained on 36,903 images from nine public sources, quantized with Brevitas to 4-bit weights and activations (8-bit stem and head), exported to QONNX and compiled with FINN into a streaming dataflow accelerator. After quantization-aware training (QAT) the model keeps mAP50 of 0.984 at close range and 0.988 at mid range, and the error of the predicted box centre is unchanged from the floating-point model. On the board the accelerator runs at 187.5 MHz, processes a frame every 5.94 ms (168 FPS) with a single-frame latency of 22.6 ms, and uses 30% of the LUTs, 79% of the block RAM and 13% of the DSP slices of the ZCU102. Its output is bit-exact against simulation: 0 of 3,744,000 INT21 values differ over 60 real frames. With a USB 3.0 global-shutter camera, the live chain (camera $\rightarrow$ accelerator $\rightarrow$ decode $\rightarrow$ tracker) runs at about 113 FPS with a median latency of 35.8 ms from the driver timestamp to the aim output. We estimate that replacing the USB camera with a MIPI CSI-2 sensor streamed directly into the programmable logic, moving pre- and post-processing from the ARM cores into the FPGA, and further optimizing the detector could reduce the total latency to about 20 ms. Code, trained weights, the build recipe and the board deployment package are available at <https://github.com/beatwad/drone-detect>.

1 Introduction

Small multirotor drones are cheap, and a system that has to respond to one at close range has to see it first. In this work the vision system has one job: detect a drone that is already large in the frame (at most 10 m away), and report how far it is from the optical axis, quickly and with little jitter, to a separate pointing subsystem. The requirements that shaped the design are:

- **Latency.** The end-to-end budget from exposure to aim output is 50–100 ms. Every millisecond of delay is error against a moving target.
- **Determinism.** The spread of the latency matters as much as its median. Jitter becomes aim error directly. This argues against NPUs and GPUs, whose inference time depends on DDR bandwidth shared with the rest of the system and on scheduling inside a vendor runtime.
- **Power.** The eventual deployment part should draw about 2–5 W, which rules out a GPU at full power.
- **One large target.** We do not need general small-object detection at the horizon.

FPGAs suit this profile. A dataflow accelerator generated by FINN [1, 2] gives every layer its own hardware, keeps the weights on chip and streams activations between layers through FIFOs. No scheduler and no off-chip weight traffic are involved, so the time per frame is constant to within tens of microseconds. Recent work has shown that FINN can compile branched YOLO networks [3] and that small Zynq parts can run YOLOv3-Tiny at over 200 FPS [4]. Those papers report accelerator throughput. The rest of a working system (capture, transfer, decoding, tracking, and how far the toolchain’s estimates can be trusted) gets less attention, and in our experience that is where most of the time went.

Contributions.

1. A full chain, from PyTorch training to a live camera on a ZCU102, for close-range drone detection: about 113 FPS and a median of 35.8 ms from the driver timestamp to the aim output, with the accelerator at 168 FPS / 22.6 ms (Sections 5–9).
2. Numerical verification of every step from the quantized PyTorch model to silicon, ending in a bit-exact on-board comparison. We also show why random test inputs hide quantization mismatches by six orders of magnitude (Section 7).

3. Measured calibration factors for FINN’s resource estimates, confirmed on two different networks to within 3.2%. The factors show that block RAM, not logic, decides whether the design fits (Section 6).
4. The root cause of a $2.4\times$ throughput shortfall: FIFO sizing on skip connections around a whole-frame operator. We found it with a whole-design Verilator model that matches the board’s frame interval to within 64 cycles, and the fix transferred to silicon as predicted (Section 8).
5. Software measurements for the ARM side of the chain, including how Python’s garbage collector, not the OS scheduler, set the worst-case latency (Section 9).

2 Related Work

Dataflow QNN accelerators. FINN [1] and FINN-R [2] compile quantized neural networks into a pipeline of streaming hardware layers. Each layer’s parallelism is set by its processing elements (PE) and SIMD lanes, a choice called *folding*. Brevitas [5] provides quantization-aware training (QAT) in PyTorch, and QONNX [6] represents arbitrary-precision quantized graphs as ONNX with explicit **Quant** nodes. Brevitas $\rightarrow$ QONNX $\rightarrow$ FINN is the toolflow used here. The quantization arithmetic follows the integer-only inference formulation [7, 8].

YOLO on FPGAs with FINN. LPYOLO [9] deployed a 4-bit TinyYOLOv3 face detector on a Zynq-7020 (PYNQ-Z2) at 18FPS and 2.4W. Calì et al. [4] reached 208FPS for YOLOv3-Tiny on a Zynq-7020 at 200MHz and 2.55W. They balanced the folding against the device’s resources, and reported that most of the board power is the processing system (ARM) and DDR at idle rather than the fabric. Danilowicz and Kryjak [3] compiled a branched, W4A4 YOLOv8n through FINN on a ZCU102. Their detector runs at 195.3FPS at 300MHz on a 320×192 input, and the whole system, with SORT [10] on the ARM cores, runs at about 24FPS. Our network is a retrained version of the topology that their bitstream actually computes (Section 5), and we built it with their FINN fork.

Vision-based counter-UAS. Benchmarks such as Anti-UAV [11] and the Drone-vs-Bird challenge [12] address detection and tracking of drones, often small and distant, and the confusion with birds. A recent survey [13] covers the field across sensor modalities. Our setting is different: the target is close, large and single, and the constraint that dominates is latency on an embedded platform.

3 Problem Setting and Optics

A 30cm drone at 10m subtends $2 \arctan(0.15/10) = 1.72^\circ$. If a 1920×1080 frame from a 60° lens is *resized* to a 416-pixel input, the drone covers about 12 pixels. That is at the detection floor we measured for a stride-8 head: 33% miss rate for 8–12px boxes, against 2.5% above 64px. A 416×416 *centre crop* of the same frame keeps the native angular resolution, and the drone covers about 55px. The crop has a second advantage: it is aligned with the boresight, so the offset from the crop centre is the pointing correction, with no coordinate transform.

We label every training image with a distance *regime* by the area of its largest box: *close* ($\geq 10\%$ of the frame), *mid* (1–10%) and *long* ($< 1\%$). Mid range turned out to be as good as close range (on an earlier YOLOv5n baseline, mAP50 0.992 mid against 0.989 close; only long range dropped, to 0.904). So the optical design does not need a telephoto lens. It only needs to keep the target out of the long regime, and a $30\text{--}45^\circ$ lens with a centre crop does that at 10m. The cost is a narrow search cone ($10\text{--}14^\circ$ effective field of view), and initial acquisition is an open problem (Section 11).

On the board the camera’s 1280×720 mode is used. We registered each capture mode against the full 1920×1200 frame (SIFT + RANSAC) and found that this mode is a native centre crop of the sensor, not a downscale. The 192×320 network input is then cut from it at native resolution.

4 Data and Training

Dataset. No public dataset targets drones at 10m, so we merged nine open sources (two from Kaggle, seven from Roboflow Universe) into 36,903 images (29,609 train, 7,294 validation): 8,878 close, 9,831 mid, 13,059 long and 5,135 with no drone. One planning assumption was wrong: a source described as long-range had a median box area of 33% of the frame, so close-range data was available from the start. Most of the cleaning effort was aimed at stopping validation from measuring memorization:

- *Near-duplicate leakage.* Many sources are video frames. We split 80/20 by whole clusters of near-duplicates, stratified by source and regime. In the first two sources only 2,891 of 5,345 images were unique clusters.

- *Degenerate perceptual hashes.* On flat sky, perceptual hashes of unrelated images collide at Hamming distance 0–6. We used pHash for recall and then confirmed matches with pixel and gradient normalized cross-correlation on thumbnails.
- *Augmented copies.* Roboflow exports contain up to six rotated copies of each photo. We kept one per base name, but only for sources that were actually augmented: in the others, the base name is not unique.
- *Unreachable boxes and dirty negatives.* We dropped images in which *every* box was below 12 px at network input. An early “empty” set turned out to be full of unlabelled drones, so we re-checked it by eye.

Mosaic augmentation at probability 1.0 raised the false-alarm rate on empty frames by about 12 points relative to 0.5, and validation mAP did not show it. We train with `mosaic=0.5`.

Metric. For aiming, the quantity that matters is where the box centre falls, not how tightly the box fits. Alongside per-regime mAP we report *centre error*: the distance between matched predicted and ground-truth centres, divided by the image diagonal. Quantization damage is often invisible at IoU 0.5. In Table 1, close-range mAP50 *rises* under post-training quantization while mAP50-95 falls 3.9 points.

Activations must be trained in, not swapped in. FINN implements activations as multi-threshold units. This works for monotone functions that become piecewise constant after quantization, which in practice means ReLU and ReLU6. The original plan was to train with SiLU and switch to ReLU before QAT. On our earlier YOLOv5n model, putting ReLU into the SiLU-trained weights dropped mAP50 from 0.963 to 0.078: about 49% of pre-activations are negative, and ReLU zeroes them in each of 57 layers. A model *trained* with ReLU from the start cost 2.1 points of mAP50-95 and nothing in mAP50 or centre error. The same lesson applies to the activation range (Section 5): a plain-ReLU model exceeded 6 in all 39 convolution blocks, with a median maximum of 21.7, so the final model is trained with ReLU6 [14] from the start.

5 Model and Quantization

Topology. We started from the YOLOv8n network compiled by Danilowicz and Kryjak [3]. A reachability analysis of their published QONNX graph showed that only the stride-8 head is connected to the output. 86 of its 269 nodes, including 22 of 63 matrix-vector units (MVAUs), compute nothing, and so does the whole bottom-up PAN path. The live subgraph is YOLOv8n layers 0–15 (backbone, SPPF and the top-down FPN [15]) plus one *legacy* detection head ($3 \times 3 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ convolutions) at stride 8. We rebuilt it as an Ultralytics [16] model definition (“yolov8n-P3”) and checked that the multiset of its 41 convolution weight shapes matches the graph. With one class the network has 1,610,337 parameters. Its output is a 24×40 grid with 65 channels: 64 distribution-focal-loss (DFL) logits [17], i.e. 16 bins for each of the four box sides, and one class logit. Removing the dead logic halves FINN’s estimate (LUT 126,200 $\rightarrow$ 58,609, BRAM18 1,090 $\rightarrow$ 507). The unpruned design would not fit the device.

Precision and the common activation range. The precision map is not uniform: the stem and the six head convolutions use 8-bit weights and unsigned 8-bit activations, and the other 34 convolutions use INT4 weights (per channel) and UINT4 activations. Every activation quantizer has the same range [0, 6], with scale 6/15 at UINT4 and 6/255 at UINT8. This common range matters for FINN. During streamlining, FINN moves scalar multiplications through the graph until they merge into thresholds, and $a x + b y$ can be rewritten as $(x + y) a$ only if $a = b$. If the branches entering a concatenation or residual add have different scales, a floating-point multiplication remains in front of the join and nothing downstream of it becomes hardware. With one activation range, every join shares a scale by construction.

Training and results. The float model is trained from COCO-pretrained YOLOv8n weights for 100 epochs at 320 px (SGD, batch 192). QAT runs 30 epochs from the float checkpoint. Table 1 shows the results. QAT recovers 96% of the close-range mAP50-95 lost by post-training quantization and 86% of the long-range mAP50, and the close-range centre error is unchanged (14.03 px vs. 14.10 px). The ReLU6 clamp itself costs almost nothing against plain ReLU (close 0.9855 vs. 0.9878, long 0.8691 vs. 0.8643). We build at 192×320 instead of square, because the design is constrained by BRAM (Section 6); centre error at this shape measured 0.0119.

Export. The exported QONNX graph matches the reference graph operator for operator: 41 Conv, 10 Concat, 6 Split, 3 MaxPool and 2 Resize. To get there, `Tensor.chunk` in C2f has to be replaced by `torch.split` at export. Otherwise tracing produces 12 `Slice` nodes with no hardware equivalent.

Table 1: Accuracy of yolov8n-P3 on the validation set, by distance regime. Centre error is normalized by the image diagonal (close regime).

model	close		mid	long		centre err.
	mAP50	mAP50-95	mAP50	mAP50	mAP50-95	close
Float, ReLU6	0.9855	0.7128	0.9893	0.8691	0.4848	0.0139
W4A4 PTQ	0.9880	0.6737	—	0.7663	0.3540	—
W4A4 QAT	0.9835	0.7099	0.9879	0.8514	0.4733	0.0139

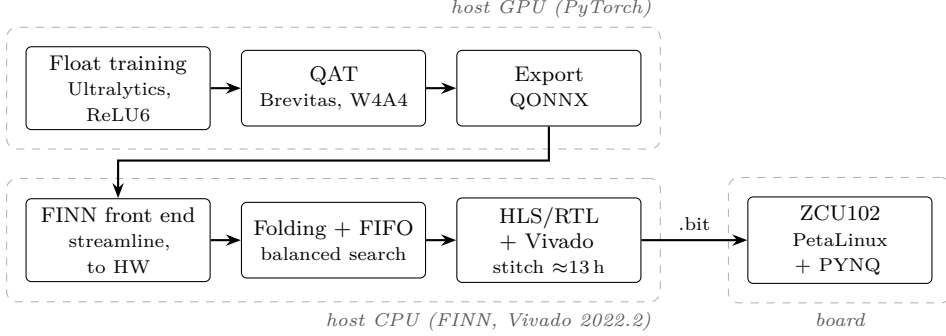


Figure 1: Toolflow. Everything up to the QONNX export is independent of the board.

6 Compiling with FINN

Figure 1 shows the toolflow. Before the YOLOv8 network we brought up two smaller YOLOv5-derived networks through FINN: a branch-free 357k-parameter network (“pico”) and a reduced YOLOv5n with the full FPN (“n_eighth”, 446k parameters). Several of the findings below come from those runs.

Branched graphs compile. Earlier FINN YOLO deployments on the Zynq-7020 removed the FPN, and we had read that as a toolchain limitation. It is not: those designs dropped the FPN for lack of resources on the small device. Joins need two things. The first is a shared quantization scale (Section 5). The second is the transformations that move multiplications across joins and forks (`MoveMulPastJoinAdd`, `MoveMulPastJoinConcat`, `MoveLinearPastFork`). These are not part of FINN’s default `Streamline()` sequence. With a custom step that applies them until nothing changes, the number of stranded multiplications in `n_eighth` fell $60 \rightarrow 17 \rightarrow 8 \rightarrow 7$, and every concatenation and add became a hardware layer. None of FINN’s code trees was usable on its own for the YOLOv8 build. The development branch fails partitioning, and the reference fork’s HLS library crashes the compiler front end on every `StreamingConcat`. The working combination is the authors’ fork with one header (`concat.hpp`) taken from the development branch.

Folding has to be balanced. FINN’s `target_fps` option makes every layer *at least* as fast as the target but does nothing to match layers to each other. On pico the first convolution ended up six times slower than the layers feeding it, and automatic FIFO sizing asked for a 289,032-word buffer, beyond Vivado’s maximum of 32,768. Following Calì et al. [4], we search for the smallest cycles-per-frame that still fits the device, reusing FINN’s own folding pass so that its divisibility constraints hold. Table 2 shows the effect. The search has two traps. Resources must be estimated *after* FINN’s bit-width minimization pass, which reduces LUT by about $3\times$. And resources are not monotonic in folding: fully folded layers push narrow, deep weight memories into LUTRAM, so a binary search is not valid.

Calibrating FINN’s estimates. Table 3 compares FINN’s estimates with post-route Vivado numbers for three builds. When the multiply-accumulates run in LUTs (HLS MVAU, W8A8), Vivado needed $2.73\times$ the estimated LUTs. Placement was refused, and the carry chains (CARRY8, which FINN does not model) reached 104% of the device. With 4-bit weights and the RTL MVAU, which packs several MACs into each DSP48E2 slice [18], the LUT factor drops to $1.44\times$ and CARRY8 to 4.6%. In FINN, the RTL MVAU is selected only when thresholds are standalone layers [18]. Block RAM is under-estimated by $2.86\times$ because FINN’s per-layer estimate does not include FIFOs. We measured these factors on the pruned reference YOLOv8n and then used them, unchanged, to predict our own drone network: LUT, BRAM and DSP came within 0.8%, 3.2% and 2.3% of the prediction. The design is constrained by BRAM: it runs out first, at 79%

Table 2: Balanced folding against FINN’s `target_fps` on the branch-free pico network (FINN estimates, 100 MHz).

configuration	FPS	LUT (est.)	deepest FIFO
<code>target_fps=30</code>	32	19,472	289,032
balanced	572	119,128	1,627

Table 3: FINN estimate vs. Vivado (post-synthesis for pico, post-route otherwise), ZCU102. “Ref.” is the pruned reference YOLOv8n; “Ours” is yolov8n-P3 W4A4 at 192×320 , 100 MHz.

design	resource	FINN est.	Vivado	ratio
pico, W8A8, MACs in LUT	LUT (logic)	118,916	324,702 (118%)	2.73
	CARRY8	—	35,715 (104%)	—
	BRAM18	255	372	1.46
Ref. YOLOv8n, W4A4, MACs in DSP	LUT	58,609	84,364 (30.8%)	1.44
	BRAM36	253.5	725.5 (79.6%)	2.86
	DSP	315	359 (14.3%)	1.14
Ours, W4A4, MACs in DSP		predicted	Vivado	error
	LUT	~82,900	82,222 (30.0%)	0.8%
	BRAM36	~714	691 (75.8%)	3.2%
	DSP	~342	334 (13.3%)	2.3%

of the device against 30% of LUTs and 13% of DSPs. The shipping build uses 720 BRAM36 (25.9 Mbit), $3.7\times$ the 7.02 Mbit theoretical weight footprint.

Practical failure modes. Several toolchain problems each cost hours or days. We list the ones that recur, since anyone repeating this flow is likely to hit them:

- *Vivado IP catalogue overflow (BD 5-336).* FINN registers a separate IP repository path for each generated core, about 700 of them. By the time the partition wrapper is instantiated its own definition has been evicted, and the build fails 23–36 h in. Registering one directory of symbolic links to all cores once, before the first cell is created, fixes it. One core is easy to miss: `memstream`, which streams weights into the RTL MVAUs, is referenced as `$FINN_ROOT/...` rather than by an absolute path. A filter that collects only absolute paths drops it, and the build then fails near the end, while generating child IP.
- *Silent HLS failures.* Parallel Vitis HLS runs race and leave one of 41 layers unpackaged, in both of our builds. FINN logs “using pre-existing IP”, and the build fails hours later at stitching. We now check for each core’s `component.xml` after IP generation.
- *Quadratic stitching.* Instantiating cell i in the block design takes $5.5 + 0.32i$ seconds, so the 946-cell design took 12 h 56 min to stitch. Synthesis, place and route took 42 min.
- *Missing driver.* Because the bitstream step never returned normally, FINN did not emit its PYNQ driver. We rebuilt the parent graph from the three partitions left on disk. The last per-channel scale and bias stay outside the accelerator (scales are about 10^{-4}) and must be applied before the sigmoid.

7 Verifying the Numerical Path

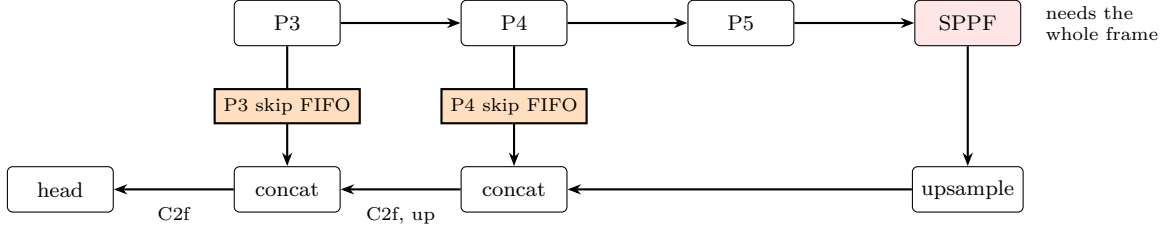
Each step from the quantized PyTorch model to the board was compared against the previous one (Table 4). The only real numerical event is a rounding convention. Brevitas rounds x/s half-to-even, while FINN’s `MultiThreshold` compares against thresholds folded into floating point, so a value exactly on a threshold can go either way. From `convert_to_hw` onwards the graphs are bit-exact.

Three things produced false alarms during verification:

1. *Random inputs hide mismatches.* Disagreement is bimodal per frame: either a frame matches exactly, or one early least-significant-bit flip cascades and shifts almost the whole output tensor (62,150 of 62,400 values). Flat sky puts many activations exactly on thresholds, while random noise almost never does. The maximum difference was 4.8×10^{-6} on random data and up to 14.47 on real frames.
2. *The confidence threshold is a discontinuity.* A step accurate to 5×10^{-6} “gained” a detection at the threshold. Detections should be compared above `conf + 0.05`.
3. *Sampling bias.* Validation lists are ordered by source, so the first n images cover one or two of the nine sources. A strided sample (every k -th image) is needed, which covers all sources in proportion.

Table 4: Discrepancy at each step of the path from PyTorch to silicon, on real validation frames.

step	max. difference to previous	verdict
Brevitas export vs. QAT model	± 1 activation step, data dependent	rounding convention
FINN front end (<code>Quant</code> $\rightarrow$ <code>MultiThreshold</code>)	4.8×10^{-6}	float noise
Streamlining (229 $\rightarrow$ 136 nodes)	$\leq 7\%$ of one step, 570 of 62,400 values	float noise
<code>convert_to_hw_layers</code>	0	bit-exact
Board vs. simulation (60 frames)	0 of 3,744,000 values	bit-exact

**Figure 2:** The two FPN skip branches (highlighted) run around SPPF, whose latency exceeds one frame interval. Sized at one tensor, they let only one frame sit between fork and join.

On 250 frames per regime, the matched-detection count changed by at most one in 750 frames, and the centre error changed by at most 0.07px with no consistent sign. Accuracy measured on the host therefore holds on the hardware. The on-board check uses a 0.05 LSB tolerance: a real error is 1.0 LSB and float32 noise at these magnitudes about 0.008 LSB. A host-side self-test with positive and negative controls (0.000 and 0.998 LSB) makes sure the comparison itself works.

8 The 2.4 $\times$ Throughput Gap: Skip FIFOs One Tensor Deep

The first bitstream (100 MHz) gave a single-frame latency of 42.2 ms and a steady-state interval of 26.2 ms, i.e. 38 FPS. FINN estimated 90.4 FPS. DMA was not the cause: about 185 kB per frame in each direction is about 7 MB/s.

Whole-design simulation. We compiled the stitched design’s Verilog with Verilator [19], replacing its 173 Xilinx AXI-Stream FIFOs with a model whose depth can be set per instance at run time and which reports peak occupancy. The model reproduces the board: latency 4,207,300 vs. 4,223,578 cycles, interval 2,623,215 vs. 2,623,279. A two-frame run takes minutes and needs neither Vivado nor FINN. Every HLS layer runs exactly as long as FINN estimates, so the gap had to come from the interaction between layers.

Cause. The FPN has two long skip branches, from P3 and from P4 to their concatenations. The other branch of each fork goes through P5 and SPPF, which cannot emit a row until it has seen the whole frame, so that path takes more than one frame interval (Figure 2). The skip FIFOs had been sized to exactly one tensor. Only one frame fits between fork and join, the fork blocks, and the pipeline processes one frame at a time. FINN’s `largefifo_rtlsim` strategy caps every FIFO at its tensor size during the sizing simulation. A skip that needs more saturates at the cap, the sizing run itself executes one frame at a time, and the “measured” depth is the cap. The cap guarantees no deadlock, but says nothing about throughput. We expect the same in any FINN network with a skip around a whole-frame operator (SPPF, global pooling). The symptom is a join-input FIFO filled to exactly 100% of its tensor.

Fix. The required depth is roughly the long branch’s latency divided by the interval (2.4 tensors on P3, 1.7 on P4). We deepened the P3 skip from 61,440 to 159,744 words and the P4 skip from 30,718 to 55,294. In simulation the interval became 1,114,135 cycles (89.8 FPS at 100 MHz) for about 30 extra BRAM36. Latency stayed at 42 ms in every configuration. It is structural: the post-SPPF half cannot start a frame until the pre-SPPF half has finished it.

Clock. The reported critical path (8.2 ns, about 121 MHz) turned out to be the reset net: one flip-flop fanned out across the die, 97.8% of its delay in routing. The worst data path is 5.45 ns (about 183 MHz). We rebuilt with the two FIFO depths changed and a 5 ns target. An offline check confirmed that 335 of 337 FIFO edges were unchanged. Timing closed at 187.48 MHz (+0.55 ns), with 82,907 LUT (30.3%) and 720

Table 5: Accelerator on the board, from batch-size sweeps. Latency is the time for a single frame; interval is the time per additional frame in a large batch, i.e. the slope of total time vs. batch size.

	100 MHz, original	187.5 MHz, FIFO fix	predicted
interval	26.2 ms (38 FPS)	5.94 ms (168 FPS)	~5.9 ms
single-frame latency	42.2 ms	22.6 ms	~22 ms
bit-exact check	pass	pass	—

BRAM36 (79.0%, +29 against the simulation’s estimate of +30). Table 5 shows the board measurements: the fix found in simulation transferred to silicon as predicted.

Single-frame latency is about 3.8 times the interval. Lowering it further needs the layers on the critical path folded more aggressively, not deeper buffers.

9 The Live System

Platform. We built Linux for the board with PetaLinux 2022.2, run in a container because the host OS is far newer than the Yocto release supports. Three settings are required: the board machine name (a custom XSA otherwise yields a generic ZynqMP device tree); device-tree GPIO hogs that set the PS-GTR lane mux, without which the board has no USB 3.0 at all [20]; and the dual-role DWC3 USB driver, because the host-only configuration fails to link on the 5.15 vendor kernel. No PYNQ image exists for the ZCU102 and the image has no compiler. None of PYNQ’s native extensions are on our path, however, so PYNQ [21] 3.0.1 is rebuilt on the host as a pure-Python wheel. The bitstream is loaded at run time, not baked into the boot image.

Capture. The camera is a USB 3.0 UVC global-shutter camera on an AR0234 sensor. As noted in Section 3, its 1280×720 mode is a native centre crop of the sensor. Nominally 200 FPS, it delivers about 249 FPS under auto exposure, with no repeated frames among 370 checked. We capture YUYV rather than MJPEG. The crop then happens before colour conversion, only the 192×320 window is converted, and there is no JPEG decoding time that depends on scene content and would add jitter. A raw V4L2 capture thread keeps only the newest frame and converts it in C (1.6 ms, against 14.7 ms in NumPy; identical to OpenCV over all 16.7 M YUV triples). The `uvccvideo` timestamp marks the arrival of the frame’s first USB packet, not the start of exposure, and the camera spreads each frame over the whole frame interval (5.3 ms at this mode). All ages below are therefore lower bounds: glass-to-aim latency equals exposure plus age at aim.

Decoding. Each 21-bit output value is packed into three bytes of the DMA buffer (about 187 kB per frame). FINN’s reference unpacking goes through hexadecimal strings and took 25.6 s per frame. A NumPy version took 6.4 ms and a C version 2.6 ms. We finally avoid most of the unpacking. Confidence is a monotone function of a single channel, so $\text{conf} \geq 0.25$ is equivalent to $\text{logit} \geq \ln(0.25/0.75) = -1.0986$. A C routine reads only the class channel of all 960 cells directly from the DMA buffer and unpacks just the cells that pass, usually about ten. The DFL softmax and expectation are computed for those cells only. The result is bit-identical to full decoding, and matches Ultralytics’ reference decode to 6.1×10^{-5} px in box coordinates and 1.2×10^{-7} in confidence. It takes 2.1 ms on the board.

Tracker. The system aims at one target, so the tracker is single-target by construction. It works on pre-NMS boxes:

1. Seed with the confident box nearest to the boresight.
2. Cluster the boxes with IoU > 0.55 to the seed.
3. Fuse the cluster into one measurement by weighted boxes fusion [22].

Table 6: Live chain on the board: median age at aim (driver timestamp to tracker output) and loop rate as optimizations were added.

configuration	age at aim	loop rate
first live run (100 MHz bitstream, no pipelining)	64.6 ms	18.6 FPS
187.5 MHz bitstream + pipeline depth 3	43.6 ms	85.7 FPS
INT21 unpack in C (7.0 $\rightarrow$ 2.6 ms)	38.4 ms	99.5 FPS
colour conversion in the capture thread	37.7 ms	103.4 FPS
decode directly from the DMA buffer	35.8 ms	113.0 FPS

The fused measurement z (centre z_c , size z_s) updates an α - β filter [23], i.e. a constant-velocity filter with fixed gain, which is what a Kalman filter converges to in steady state:

$$\hat{c} = c + v \Delta t, \quad r = z_c - \hat{c}, \quad c \leftarrow \hat{c} + \alpha r, \quad v \leftarrow v + \frac{\beta}{\Delta t} r, \quad s \leftarrow (1 - \gamma)s + \gamma z_s, \quad (1)$$

where

c	smoothed estimate of the drone’s centre (x, y) , px;
v	estimated velocity of the centre, px/s;
s	smoothed box size (width, height), px;
Δt	time between the driver timestamps of consecutive frames, s;
$\hat{c}$	predicted centre for the current frame, px;
z_c, z_s	centre and size of the fused measurement z , px;
r	residual: how far the measurement is from the prediction, px;
$\alpha = 0.6$	position gain: the share of the residual applied to the centre;
$\beta = 0.2$	velocity gain: the share of the residual applied to the velocity;
$\gamma = 0.5$	exponential smoothing factor for the box size.

Values on the right of $\leftarrow$ are from the previous frame. Δt comes from driver timestamps so that the filter steps from capture time, not processing time. A gate rejects a measurement whose IoU with the predicted box is ≤ 0.30 . After 0.25 s of consecutive misses the filter re-initializes from the longest recent run of mutually consistent measurements, so a single outlier cannot restart the track.

The aim offset and its distance relative to the target size are

$$(d_x, d_y) = c + v \tau - o, \quad dist = \frac{\|(d_x, d_y)\|}{\min(s)/2}, \quad (2)$$

where

$c + v \tau$	where the drone’s centre will be after time τ , px;
o	the crop centre (160, 96), px. The crop is centred and aligned with the aiming axis, so o is where the system is pointing now, and (d_x, d_y) is the correction needed to bring the aim onto the target;
$\tau = 36$ ms	lead time: the drone keeps moving while the frame is processed (at 10 m/s it travels 36 cm in 36 ms, more than its own size). It is the measured median latency from driver timestamp to aim output; exposure, actuator response and projectile time of flight are still unknown and belong here too;
$\min(s)/2$	half the short side of the smoothed box, px;
$dist$	the offset as a fraction of that half-side: at $dist = 1$ the aim point lies on the edge of the box, whatever the drone’s apparent size.

Whether a shot hits depends on where it lands on the drone itself, so a threshold in pixels or as a fraction of the frame would be too strict for a near drone and too lenient for a distant one. The “centred” decision uses hysteresis on $dist$ (centred below 0.5, not centred above 1.0, unchanged in between) and a 0.10 s debounce. All thresholds are in seconds, not frames, so they do not change meaning when the frame rate changes. They are placeholders until real tracks are available. The tracker takes 0.1–1.8 ms per frame.

Pipelining and the CPU side. We program the DMA engines directly and keep up to three frames in flight in the accelerator. Throughput triples at the same frame age (43.6 vs. 44.4 ms). Table 6 lists the optimizations in the order we applied them. Each row was measured on 600 to 70,000 live frames.

The garbage collector. A 24-minute run (146,900 frames) showed stable medians but at least one stall of about 240 ms. It appeared in three independent stages at once, and the capture thread stopped too, so the whole process had paused. For aiming, a gap that long means the target is lost. With collection disabled, a single explicit `gc.collect()` before the loop took about 190 ms, the size of the stall. A full collection walks

Table 7: Ten-minute runs, 70,000 frames each. “Aim gap” is the time between consecutive aim outputs.

	GC default	<code>gc.disable()</code>	<code>gc.freeze()</code>
frames over 60 ms	6 (one stall)	0	0
age at aim, max	278.4 ms	46.9 ms	46.8 ms
aim gap, max	247.5 ms	26.1 ms	20.0 ms

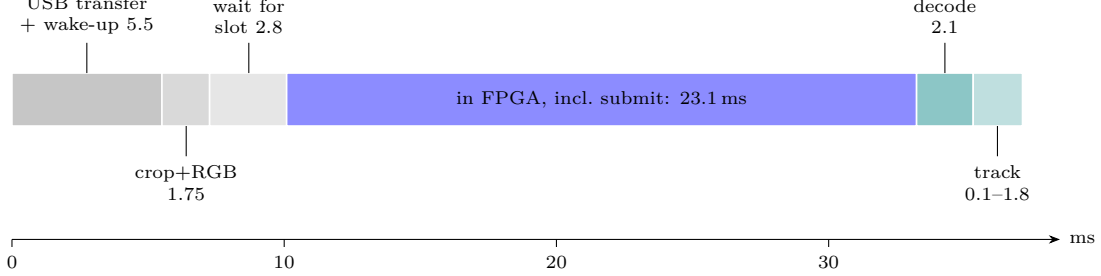


Figure 3: Median per-stage latency of the live chain (70,000 frames), from the driver timestamp to the aim output. Exposure precedes the timestamp and is not included.

every object that PYNQ, pydantic and NumPy created at start-up. Calling `gc.freeze()` [24] before the camera starts moves those objects into a permanent generation. The collector keeps running, but only on new objects (Table 7). Real-time scheduling priority was not needed.

Where the time goes. Figure 3 breaks down the final configuration. The accelerator accounts for 62% of the age at aim, frame delivery for 28%, and the rest of the CPU work for about 12%. We estimate that only 1–2 ms remain to be gained in software.

10 Results and Comparison

Live demonstration. The board runs the chain stand-alone: a systemd service starts it at boot, and a user LED on the board is lit while a drone is in the frame. Figure 4 shows two frames from a video of the running system, with the camera pointed at a drone photograph on a monitor. The LED is lit while the drone is visible and goes off when the camera is covered by a hand. This shows the full chain working on the board in real time. It is not an accuracy measurement: the target is an image on a screen, not a real drone (Section 11).

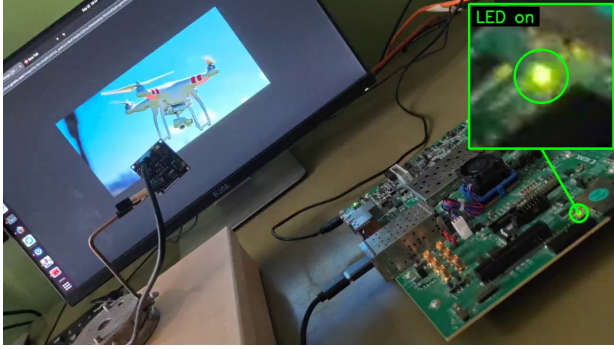
Table 8 compares our system with the two closest FINN-based YOLO systems. At the accelerator level, the comparison is roughly a tie: our clock is lower and our input is smaller. At the system level, our chain runs 4–5× faster with the same structure (an accelerator in the FPGA and Python on the ARM cores). The difference comes from what surrounds the network: pipelining, decoding in place in the DMA buffer, and controlling the garbage collector. Two caveats: our network has one detection head and thresholds before decoding, whereas the reference runs three heads and full NMS plus SORT; and our latency is measured from the first USB packet, not from exposure. The latency for Cali et al. is our own sum of the stage times in their Table 10.

Power. Vivado’s post-route estimate for the 187.5 MHz design is 6.54 W, of which the ARM processing system accounts for 2.74 W. At 100 MHz the whole fabric was 1.63 W of 5.10 W. This matches the observation of Cali et al. [4]: the processing system, not the fabric, dominates power. An architecture that moves capture, decoding and tracking into the FPGA would lower power and latency jitter together. Board power under load has not been measured yet.

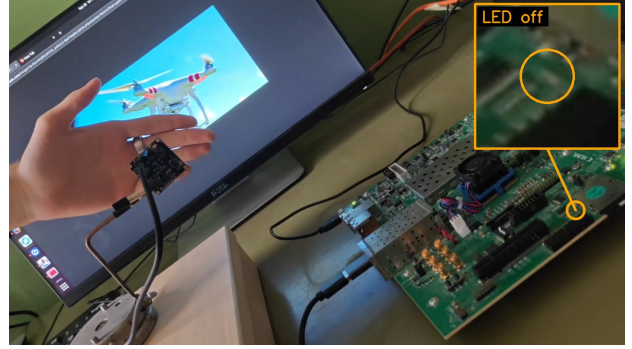
11 Limitations

This is a proof of concept, and its main gaps are:

- **No real footage through the deployment lens.** All accuracy figures come from varied web imagery. The deployed input is a fixed-scale crop through one lens, so the drone-size and context distributions differ from training. The tracker has run only on synthetic tracks and on drone images shown to the camera, and all its thresholds are placeholders.



(a) Drone in view: LED on.



(b) Camera covered: LED off.

Figure 4: The live system on the ZCU102. The USB camera (left, on the stand) faces a drone photograph on a monitor. The marked user LED on the board is lit while the detector and tracker report a drone in the frame (a), and goes off when the camera is covered (b).

Table 8: Comparison with FINN-based YOLO systems.

	accelerator	full system	system latency
Danilowicz & Kryjak [3] (YOLOv8n, ZCU102, 300 MHz)	195.3 FPS	~24 FPS	not reported
Cali et al. [4] (YOLOv3-Tiny, Z7020, 200 MHz)	208.7 FPS	30 FPS	~70 ms
This work (yolov8n-P3, ZCU102, 187.5 MHz)	168 FPS	~113 FPS	35.8 ms (median)

- **Birds and aircraft.** On 1,003 hard-negative images (birds and aircraft framed as targets), the model fires on 10.1% at confidence 0.6 and 6.1% at 0.7, scoring 0.67–0.86. That is the same band as mid-range drones, whose recall at those thresholds is 79.3% and 62.4%. No threshold separates the two, and temporal confirmation does not help, because a bird gives a steady track. Explicit bird and aircraft classes (two more output channels, nearly free in hardware) and motion cues from the tracker (e.g. hovering, wing-beat oscillation, straight-line flight) are the planned levers, as the Drone-vs-Bird literature also suggests [12].
- **No hardware trigger.** The exposure time is unknown to the software, so the reported latencies are lower bounds.
- **Acquisition.** A 10–14° effective field of view needs platform slew, an external cue or a second wide-angle sensor to find the target in the first place. In an aim-assist setting this can also be left to the operator, who brings the drone into the field of view by hand before the system takes over.
- **USB and Linux on the critical path.** USB keeps the ARM cores and DDR powered and routes every frame through the Linux scheduler. The intended deployment architecture streams a MIPI sensor into the FPGA, with preprocessing, decoding and tracking also in the FPGA.
- **Deployment hardware.** The ZCU102 is a development board, and at 720 BRAM36 the design does not fit a much smaller FPGA as built. One path to a 2–5 W product is to split the network. The backbone and FPN, which are unlikely to change, would become a fixed-function ASIC. Only the detection head would stay in a small low-power FPGA (e.g. a Lattice FPGA), so that it can still be fine-tuned on real footage after deployment. The head is a natural cut: it is the six 8-bit convolutions, about 152k weights (1.2 Mbit), and its input is a single $24 \times 40 \times 64$ UINT4 tensor, about 30 kB per frame. The cost is the ASIC’s non-recurring engineering and a backbone frozen at tape-out.

12 Conclusion

A W4A4 YOLOv8n-derived detector, compiled with FINN, runs bit-exact on a ZCU102 at 168 FPS with 22.6 ms latency, and a live camera chain built around it reaches about 113 FPS with a median of 35.8 ms to the aim output. Accuracy and quantization took the first two weeks and went roughly to plan. Most of the cost came after export: matching quantization scales at joins, balanced folding, calibrating resource estimates, FIFO depths on skip connections, and toolchain failures that report success. Four practices helped: measure on real data rather than random inputs; choose a metric that reflects the task (centre error rather than mAP); build hardware constraints (activation function, activation range) into training rather than export; and keep a fast simulation of the whole design, which here replaced a 13-hour rebuild with a few minutes and predicted the silicon result exactly. The next steps are real footage through the chosen lens, a hardware trigger, and moving the front and back ends of the chain into the fabric.

Intended use. The system is designed to protect against hostile drones at short range. It outputs a target offset and a centring flag. Any decision to act on that output belongs to a separate subsystem with its own safeguards, and that subsystem is outside the scope of this work.

References

- [1] Yaman Umuroglu, Nicholas J. Fraser, Giulio Gambardella, Michaela Blott, Philip Leong, Magnus Jahre, and Kees Vissers. FINN: A framework for fast, scalable binarized neural network inference. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, pages 65–74, 2017. doi: 10.1145/3020078.3021744.
- [2] Michaela Blott, Thomas B. Preußer, Nicholas J. Fraser, Giulio Gambardella, Kenneth O’Brien, Yaman Umuroglu, Miriam Leiser, and Kees Vissers. FINN-R: An end-to-end deep-learning framework for fast exploration of quantized neural networks. *ACM Transactions on Reconfigurable Technology and Systems*, 11(3):16:1–16:23, 2018. doi: 10.1145/3242897.
- [3] Michal Danilowicz and Tomasz Kryjak. Real-time multi-object tracking using YOLOv8 and SORT on a SoC FPGA. In *Applied Reconfigurable Computing. Architectures, Tools, and Applications (ARC 2025)*, Lecture Notes in Computer Science, pages 214–230. Springer, 2025. doi: 10.1007/978-3-031-87995-1_13. arXiv:2503.13023.
- [4] Riccardo Cali, Laura Falaschetti, and Giorgio Biagetti. Optimized implementation of YOLOv3-Tiny for real-time image and video recognition on FPGA. *Electronics*, 14(20):3993, 2025. doi: 10.3390/electronics14203993.
- [5] Alessandro Pappalardo. Xilinx/brevitas. <https://github.com/Xilinx/brevitas>, 2023. Version 0.13.0. doi:10.5281/zenodo.3333552.
- [6] Alessandro Pappalardo, Yaman Umuroglu, Michaela Blott, Jovan Mitrevski, Ben Hawks, Nhan Tran, Vladimir Loncar, Sioni Summers, Hendrik Borrás, Jules Muhizi, Matthew Trahms, Shih-Chieh Hsu, Scott Hauck, and Javier Duarte. QONNX: Representing arbitrary-precision quantized neural networks. *arXiv preprint arXiv:2206.07527*, 2022.
- [7] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2704–2713, 2018. doi: 10.1109/CVPR.2018.00286.
- [8] Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*, 2018.
- [9] Bestami Günay, Sefa Burak Okcu, and Hasan Şakir Bilge. LPYOLO: Low precision YOLO for face detection on FPGA. In *Proceedings of the 8th World Congress on Electrical Engineering and Computer Systems and Sciences (EECSS)*, 2022. arXiv:2207.10482.
- [10] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. In *IEEE International Conference on Image Processing (ICIP)*, pages 3464–3468, 2016. doi: 10.1109/ICIP.2016.7533003.
- [11] Nan Jiang, Kuiran Wang, Xiaoke Peng, Xuehui Yu, Qiang Wang, Junliang Xing, Guorong Li, Guodong Guo, Qixiang Ye, Jianbin Jiao, Jian Zhao, and Zhenjun Han. Anti-UAV: A large-scale benchmark for vision-based UAV tracking. *IEEE Transactions on Multimedia*, 25:486–500, 2023. doi: 10.1109/TMM.2021.3128047.
- [12] Angelo Coluccia, Alessio Fascista, Arne Schumann, Lars Sommer, Anastasios Dimou, Dimitrios Zarpalas, et al. Drone vs. bird detection: Deep learning algorithms and results from a grand challenge. *Sensors*, 21(8):2824, 2021. doi: 10.3390/s21082824.
- [13] Yifei Dong, Fengyi Wu, Sanjian Zhang, Guangyu Chen, Yuzhi Hu, Masumi Yano, Jingdong Sun, Siyu Huang, Feng Liu, Qi Dai, and Zhi-Qi Cheng. Securing the skies: A comprehensive survey on anti-UAV methods, benchmarking, and future directions. In *CVPR 2025 Workshops (Anti-UAV)*, 2025. arXiv:2504.11967.

- [14] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520, 2018. doi: 10.1109/CVPR.2018.00474.
- [15] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 936–944, 2017. doi: 10.1109/CVPR.2017.106.
- [16] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics>, 2023. Version 8.3.
- [17] Xiang Li, Wenhai Wang, Lijun Wu, Shuo Chen, Xiaolin Hu, Jun Li, Jinhui Tang, and Jian Yang. Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 21002–21012, 2020.
- [18] Mirza Mrahorovic. Multi-packed DSPs for MVU/VVU layers. FINN GitHub discussion #1021, <https://github.com/Xilinx/finn/discussions/1021>, 2024.
- [19] Wilson Snyder et al. Verilator. <https://www.veripool.org/verilator/>.
- [20] AMD/Xilinx. ZCU102 evaluation board user guide (UG1182), v1.7. <https://docs.amd.com/v/u/en-US/ug1182-zcu102-eval-bd>, 2023.
- [21] AMD. PYNQ: Python productivity for Zynq. <https://www.pynq.io>, 2022. Version 3.0.1.
- [22] Roman Solovyev, Weimin Wang, and Tatiana Gabruseva. Weighted boxes fusion: Ensembling boxes from different object detection models. *Image and Vision Computing*, 107:104117, 2021. doi: 10.1016/j.imavis.2021.104117.
- [23] Paul R. Kalata. The tracking index: A generalized parameter for α - β and α - β - γ target trackers. *IEEE Transactions on Aerospace and Electronic Systems*, AES-20(2):174–182, 1984. doi: 10.1109/TAES.1984.310438.
- [24] Python Software Foundation. gc — garbage collector interface: `gc.freeze()`. <https://docs.python.org/3/library/gc.html>, 2024.